

CM Track 20: Öffentliche Sicht, Massenversand und Hintergrundläufe

Datum: 2026-08-28 **Stack:** Next.js 15, React 19, TypeScript 5.9, Supabase SDK 2.98, Stripe 20.4
Tests: 1430 → 1457 (27 neue), alle grün **Typecheck:** `tsc --noEmit` ohne Fehler · **Lint:** 0 Fehler, 20 Warnungen (Bestand) · **Build:** `next build` kompiliert fehlerfrei; die anschließende statische Erzeugung läuft lokal ohne `SUPABASE_SERVICE_ROLE_KEY` nicht durch — das ist der bekannte lokale Zustand, kein Ergebnis dieses Tracks

Zusammenfassung

Die Tracks 11 bis 19 haben geprüft, was passiert, wenn jemand etwas **anfragt**: Anmeldung, Rollen, Mandantentrennung, Eingaben, Fehlermeldungen, Exporte. Dieser Track nimmt die andere Richtung — **was ChairMatch von sich aus zeigt, verschickt und nachts tut**:

- Seiten, die ohne Anmeldung offen stehen und deren Inhalt aus einem öffentlichen Formular stammt.
- Vorgänge, die viele Empfänger auf einmal treffen und Geld kosten (Newsletter-Kampagne, SMS-Versand, Werbe-Events an Meta).
- Läufe, denen niemand zusieht (nächtlicher Cron), und Zahlen, die ohne Zuschauer entstehen (Bewertungs-Schnitt, öffentliche Kennzahlen).

Acht Befunde, alle behoben. Der schwerste ist kein Datenabfluss, sondern eine fehlende Veröffentlichungs-Schranke: **jeder konnte ohne Konto eine dauerhafte, indexierbare Geschäftsseite auf chairmatch.de anlegen.** Der zweitschwerste ist ein fehlender Riegel auf dem Newsletter-Versand: zwei Klicks auf „Senden“ bedeuteten zwei vollständige Kampagnen an dieselbe Liste.

Ein dritter Strang zieht sich durch drei Befunde: **ein Ausfall wurde als Tatsache ausgegeben.** Ein fehlgeschlagener Cron-Aufruf zählte als Erfolg, ein Lesefehler auf `reviews` wurde zu „0 Bewertungen“ — und danach in den Salon zurückgeschrieben.

Befund 1: Ein nicht freigegebener Salon hatte eine öffentliche Seite (HOCH)

Schweregrad: P1. Kein Konto nötig, dauerhaft, von Suchmaschinen erreichbar.

Track 15 hat `is_active` zum Riegel auf allen **Geldstrecken** gemacht — Termin, Mietbuchung, Mietanfrage, Verfügbarkeit, Mietsuche. Der Bericht dazu hat ausdrücklich festgehalten, was übrig bleibt: *„geblieben waren ihm die Mietsuche und der Direktlink“*. Die Mietsuche wurde dort geschlossen. **Der Direktlink blieb offen** — und das ist die Hälfte, die den Rest trägt.

`/salon/<slug>` (`src/app/(public)/salon/[slug]/page.tsx`) und `GET /api/salons/<slug|id>` haben `is_active` nie angesehen. Beide lieferten jeden Salon aus, den sie in der Datenbank fanden:

Ausgeliefert	Quelle
Geschäftsname, Beschreibung, Kategorie	Registrierungsformular
Straße, PLZ, Stadt	Registrierungsformular
Telefonnummer, Website	Anbieter-Bereich
Leistungen mit Preisen, Mitarbeitende, Mietobjekte	Anbieter-Bereich
<code>LocalBusiness</code> -JSON-LD für Suchmaschinen	daraus erzeugt

Exploit-Pfad

`POST /api/register-provider` ist **öffentlich** (`publicPrefixes` in `src/middleware.ts`), verlangt kein Konto und legt den Salon mit `is_active: false, is_verified: false` an:

```
POST /api/register-provider
{ "vn": "Max", "nn": "Muster", "em": "...@example.de", "tel": "+49...",
  "geschäft": "<bis 200 Zeichen frei wählbar>",
  "st": "...", "plz": "...", "city": "...", "kat": "...",
  "gb": true, "chair": false, "agb": true, "dsgvo": true }
```

Antwort: ein Salon-Datensatz mit einem Slug aus dem gewählten Namen. Ab diesem Moment war `https://www.chairmatch.de/salon/<slug>` erreichbar — vollständig gerendert, mit Öffnungszeiten-Widget, Buchungs-Schaltfläche und strukturierten Daten für Google.

Das Rate-Limit (5/Stunde/IP, 3/Stunde/Adresse aus Track 18) begrenzt den Durchsatz, nicht die Wirkung: eine einzige Seite reicht, um einen fremden Geschäftsnamen, eine fremde Adresse oder werbenden Text auf der Domain abzulegen. Die Freischaltung durch einen Admin in `/admin/anbieter` war damit eine Formalität **nach** der Veröffentlichung.

Die zweite Hälfte betrifft echte Kundschaft: ein Salon, den ein Admin wegen Beschwerden oder fehlender Gewerbeanmeldung **gesperrt** hat, behielt seine vollständige Profilseite. Er verschwand aus den Listen, nicht aus dem Netz. Wer über einen Link, ein Lesezeichen oder ein Suchergebnis dort ankam, sah einen intakten Salon und erfuhr erst beim Buchen, dass nichts geht.

Fix

`salonIsPubliclyVisible()` in `src/lib/salon-status.ts` — dieselbe Konvention wie `salonAcceptsBusiness()` : nur ein **ausdrückliches** `false` verbirgt, `null` bleibt sichtbar (Begründung im Kopfkomentar der Datei, unverändert seit Track 15).

- `GET /api/salons/[id]` : **404**, nicht 403 — dass es unter diesem Slug einen Eintrag gibt, ist selbst schon eine Auskunft.
- `/salon/[slug]` : `notFound()` , und `generateMetadata` erzeugt für einen solchen Salon **keinen Titel und keine Geo-Meta** mehr. Sonst stünden Name und Stadt weiterhin im `<head>` und damit in jeder Link-Vorschau.

Bewusst ohne Ausnahme für den Eigentümer. Die Seite läuft mit ISR (`revalidate = 300`); ein `cookies()` -Zugriff für die Session würde sie in dynamisches Rendern zwingen und die Zwischenspeicherung für alle kosten. Der Anbieter sieht seinen Stand im Anbieter-Bereich.

⚠ Operative Folge, die yusuf kennen muss **Ein Anbieter sieht seine öffentliche Salonseite erst nach dem Freischalten.** Vorher antwortet `/salon/<slug>` mit 404 — auch ihm selbst. Das ist dieselbe Liste wie nach Track 15: `/admin/anbieter` öffnen und bei jedem Eintrag mit Badge **SUSPENDED**, der arbeiten soll, einmal „Freischalten“ drücken. Wie viele das sind, ist von hier aus nicht lesbar — `salons` ist mit dem Anon-Key nicht abfragbar (42501 aus `is_admin_or_super`), und einen DB-Zugang haben die Agents nicht.

Zusätzlich (kein Leck, Vorsorge): `services` und `rental_equipment` standen in der Route noch auf `select('*')`. Beide Tabellen führen live nichts Schützenswertes (Spaltensonde 2026-08-27, `src/test/live-schema.ts`) — mit `*` wäre aber **jede künftige Spalte automatisch öffentlich**. Jetzt Positivlisten, wie Track 9 sie für `salons` eingeführt hat.

Befund 2: Eine Newsletter-Kampagne konnte doppelt an die ganze Liste gehen (HOCH)

Schweregrad: P1. Kosten, Zustellbarkeit, Ruf — und nicht rückholbar.

`sendCampaign()` in `src/lib/newsletter-sender.ts` hat gelesen, geprüft und geschrieben — in drei Schritten:

```
if (campaign.status === 'sending' || campaign.status === 'sent') return ... // prüfen
await supabase.from('newsletter_campaigns')
  .update({ status: 'sending' }).eq('id', campaignId) // schreiben
```

Der Riegel war damit eine **Momentaufnahme**. Zwei gleichzeitige Aufrufe — zwei offene Tabs, ein Doppelklick, ein wiederholter Request nach einem Timeout — lasen beide `status = 'draft'`, kamen beide durch die Prüfung und schrieben beide `sending`.

Folge: zwei vollständige Versandläufe. Jeder Abonnent bekam dieselbe Mail zweimal, `newsletter_sends` bekam zwei Zeilen je Empfänger, und `total_sent` wurde vom zweiten Lauf überschrieben. Der Endpunkt ist auf `maxDuration = 300` gesetzt und sendet synchron im Request — die Gelegenheit für einen Wiederholungs-Request ist keine theoretische.

Anders als eine doppelte Buchung ist das nicht korrigierbar: die Mails sind raus, und der zweite Durchgang ist genau die Sorte Signal, auf die Postfächer mit Abmeldung oder Spam-Meldung reagieren.

Fix: dieselbe Bauform wie auf den Geldstrecken (Track 16) — ein bedingtes UPDATE, das den gelesenen Zustand mitprüft, und ein Blick darauf, ob eine Zeile zurückkam:

```
const { data: claimed } = await supabase
  .from('newsletter_campaigns')
  .update({ status: 'sending', updated_at: ... })
  .eq('id', campaignId)
  .eq('status', campaign.status) // ← der Anspruch
  .select('id')
if (!claimed?.length) return { ..., code: 'already_running' }
```

Dazu: `sendCampaign` gibt jetzt einen **Code** zurück (`not_found` / `already_running` / `failed`), und `POST /api/admin/newsletter/campaigns/[id]/send` macht daraus **409** statt 200. Bis hierher antwortete auch der abgelehnte Lauf mit 200 — die Oberfläche konnte gar nicht unterscheiden, ob gerade verschickt wird oder nicht.

Nicht behoben, ausdrücklich benannt: bricht der Lauf mitten im Versand ab (Vercel-Timeout), bleibt die Kampagne auf `sending` stehen und lässt sich nicht wieder aufnehmen. Das aufzulösen heißt zu entscheiden, ab wann ein Lauf als tot gilt und ob dann erneut an alle oder nur an die Restmenge gesendet wird — eine kaufmännische Entscheidung, kein Härte-Fix.

Befund 3: `/api/analytics/meta-capi` war ein offener Briefkasten ins Werbekonto (MITTEL)

Schweregrad: P2. Kein Datenabfluss, ein kaufmännischer Schaden.

Die Route nimmt Conversion-Events entgegen und schickt sie mit `META_CAPI_ACCESS_TOKEN` an die Meta Graph API. Sie **muss** ohne Anmeldung erreichbar sein — sie wird vom Browser jeder Besucherin aufgerufen. Sie hat aber auch **alles** angenommen:

- jeden `event_name`,
- jedes `custom_data` in beliebiger Größe,
- ohne jedes Rate-Limit über das allgemeine `/api/*` -Limit hinaus (60/min/IP).

```
POST /api/analytics/meta-capi
{ "event_name": "Purchase",
  "custom_data": { "value": 4999.00, "currency": "EUR", "order_id": "..." } }
```

Metas Gebotsalgorithmus optimiert auf genau diese Signale. Wer sie fälscht, steuert, an wen ChairMatch seine Werbung ausspielt und mit welchem angeblichen Umsatz jede Kampagne bewertet wird. Der Bericht im Werbemanager wird dabei zu einer Zahl, die niemand mehr gegenprüfen kann — die Ereignisse sehen aus wie echte, weil sie über denselben Weg kommen.

Dazu zwei kleinere Stellen in derselben Datei:

- **Metas Fehlerobjekt ging roh an den Aufrufer** (`{ error: 'meta_capi_error', detail: data }`) — inklusive `fbtrace_id`, Ausnahmetyp und Grund der Ablehnung. Ebenso `meta: data` im Erfolgsfall.
- **Der Stub-Zweig stand vor dem Lesen des Bodys.** Sein Kommentar behauptete „validiert Payload-Format“ — tatsächlich hat er den Body nie angesehen und auf jeden Aufruf `ok: true` geantwortet.

Fix: Positivliste der zwölf Ereignisse, die die Anwendung tatsächlich meldet; Positivliste der `custom_data` -Felder mit Längenbegrenzung (200 Zeichen je Zeichenkette, 20 Elemente je Liste); Rate-Limit 60/Minute/IP; Metas Fehlerobjekt nur noch ins Log. Die Prüfung läuft jetzt **vor** dem Stub-Zweig — damit ist sie auch ohne Meta-Zugangsdaten wirksam und prüfbar.

Befund 4: SMS-Pumping über `/api/auth/phone/send` (MITTEL)

Schweregrad: P2. Direkte Kosten bei Twilio, ohne Konto auslösbar.

Der Deckel lag ausschließlich auf der **Zielnummer**: 3 SMS pro Nummer pro 10 Minuten, gezählt in `phone_verifications`. Das begrenzt, wie oft *eine Person* belästigt werden kann. Es begrenzt nicht, wie viele Nummern ein Aufrufer durchprobiert.

Genau darauf beruht SMS-Pumping (auch „Toll Fraud“): der Angreifer besitzt oder mietet Nummernblöcke bei einem Netzbetreiber, der ihn am Zustellentgelt beteiligt, und lässt eine fremde Anwendung die SMS bezahlen. Jeder Aufruf kostet ChairMatch echtes Geld — ohne Konto, ohne Session.

Übrig blieb das Middleware-Limit: 10 Requests pro Minute und IP auf `/api/auth/phone/`, also bis zu **14.400 SMS pro Tag aus einer einzigen Quelle**, und beliebig viel mehr über mehrere.

Fix:

1. Kontingent **pro IP**, das in Stunden statt in Minuten rechnet (10/Stunde, `src/lib/rate-limit.ts`). 2. **Ländervorwahl-Positivliste** `+49 / +43 / +41`. ChairMatch ist ein deutschsprachiger Marktplatz; die Nummernbereiche, über die Pumping abgerechnet wird, liegen typischerweise außerhalb. Eine Strecke für ausländische Nummern hat die Anwendung heute ohnehin nicht.

Das IP-Limit liegt im Speicher der Instanz (bekannte Grenze, siehe `src/lib/rate-limit.ts`) — es hält die Schleife aus einer Quelle auf, nicht den verteilten Angriff. Die Ländersperre gilt unabhängig davon.

Befund 5: `publish-reviews` meldete Erfolg für fehlgeschlagene Aufrufe (MITTEL)

Schweregrad: P2. Ein stiller Ausfall in einem Lauf ohne Zuschauer.

```
for (const bookingId of uniqueBookings) {
  try {
    await supabase.rpc('publish_review_pair', { p_booking_id: bookingId })
    published++
  } catch (e) { logger.warn(...) }
}
```

`supabase.rpc()` **wirft nicht**. Es liefert `{ data, error }` — wie jede andere Abfrage des SDK. Der `try/catch` hat also nie etwas gefangen. Ein Fehler der Stored Procedure (fehlende Funktion, verletzte Bedingung, entzogenes Recht) landete still in einer Variablen, die niemand ansah, und `published++` lief trotzdem.

Die Antwort behauptete danach das Gegenteil dessen, was passiert war: `{ ok: true, published: 87 }` für 87 Aufrufe, von denen keiner durchging. Der Cron läuft nachts um 03:30 — diese Antwort ist die **einzige** Stelle, an der auffallen kann, dass Bewertungen seit Wochen nicht mehr freigeschaltet werden.

Dieselbe Datei, zweite Stelle: die Abfrage der fälligen Bewertungen las den Fehler ebenfalls nicht (`const { data: stale } = await ...`). Fiel sie aus, war `stale` leer, und der Lauf antwortete `{ ok: true, published: 0 }` — ein Ausfall der Datenbank war von „nichts zu tun“ nicht zu unterscheiden.

Fix: `error` wird gelesen; ein fehlgeschlagener Aufruf zählt in `failed`, nicht in `published`; die Antwort trägt `ok: failed.length === 0` und die Zahlen `published / failed / processed_bookings`. Ein Ausfall der Abfrage gibt **503**. Der `catch` bleibt stehen — ein geworfener Fehler (Netzwerk, abgebrochene Verbindung) ist weiterhin möglich, nur nicht mehr der einzige Weg, auf dem ein Fehlschlag ankommt.

Harness: `rpc()` gab es im Testnachbau bis hierher gar nicht — die einzige Stelle, an der Bewertungen öffentlich werden, war damit nicht prüfbar. `FakeSupabase.rpc()` / `.onRpc()` / `.rpcCalls` schließen die Lücke.

Befund 6: Ein Lesefehler löschte den Ruf eines Salons (MITTEL)

Schweregrad: P2. Datenverlust ohne Fehlermeldung, nicht wiederherstellbar.

```
async function salonReviewRatings(salonId: string): Promise<number[]> {
  const { data: reviews } = await supabase.from('reviews')... // kein error
  return (reviews || []).filter(isSalonReview)... // → []
}
```

Der Fehler wurde nicht einmal destrukturiert. Fällt die Abfrage aus — 22P02 bei einer ungültigen ID, 42501 bei einer Policy, ein Aussetzer der Datenbank — ist `reviews` null, und die Funktion gibt eine **leere Liste** zurück. Ein Ausfall war von „dieser Salon hat noch keine Bewertung“ nicht zu unterscheiden.

Beide Aufrufer haben daraus eine Tatsachenbehauptung gemacht:

- `getAggregateRatings` → `{ avgRating: 0, reviewCount: 0 }` mit Status 200. Die Salonseite zeigte „keine Bewertungen“.
- **updateSalonRating** **SCHRIEB diese Null** nach `salons.avg_rating` und `salons.review_count`. Diese Funktion läuft nach jeder gespeicherten Bewertung. Ein einziger Aussetzer in diesem Moment ersetzte damit dauerhaft 4,6 ★ aus 31 Bewertungen durch 0 aus 0 — die Ausgangswerte sind danach nirgends mehr vorhanden, und es fällt erst auf, wenn jemand hinsieht.

Dazu, in derselben Kette: `GET /api/reviews/aggregate` prüfte `salonId` nicht. `salons.id` ist eine `uuid`-Spalte, eine Nicht-UUID beantwortet PostgREST mit 22P02 — der wurde nicht gelesen, und die Antwort war ein 200 mit Schnitt 0.

Fix: `salonReviewRatings` gibt `null` für „nicht ermittelbar“ zurück. `updateSalonRating` schreibt dann **nichts** (der alte Stand bleibt stehen) und meldet `false`; `createReview` protokolliert das, die Bewertung selbst ist gespeichert und der Schnitt zieht beim nächsten Lauf nach.

`getAggregateRatings` gibt `null`, die Route macht daraus **503**. Nicht-UUID → **400** (`isUuid` aus Track 19).

Befund 7: `/api/public-stats` zählte Gelöschtes und Gesperrtes mit (NIEDRIG)

Schweregrad: P3. Ohne Anmeldung abrufbar, und die Zahlen waren falsch.

Die Route lief mit dem Service-Client, umging RLS also mit Absicht, und zählte danach **alle** Zeilen von `profiles`, `salons`, `bookings`, `reviews`.

- `profiles` enthält zur Löschung angemeldete und bereits hart gelöschte Konten (`deleted_at`, `delete_requested_at`) — mitgezählt.
- `salons` enthält jeden selbst registrierten Salon, den nie ein Admin freigeschaltet hat — mitgezählt.
- `cityList` nannte die Städte **aller** Salons. Wer wissen wollte, wo ChairMatch gerade Anbieter anwirbt, musste diese Route aufrufen, sonst nichts.

Die Route hat in der Anwendung **keinen Aufrufer** (Stand dieses Tracks); sie steht in `publicPrefixes` und ist damit von außen erreichbar.

Fix: `profiles` ohne `deleted_at` / `delete_requested_at`, `salons`, `cityList` und `categories` nur mit `is_active = true`; Städte sortiert.

Befund 8: Rohe Datenbankmeldung in `DELETE /api/upload/[id]` (NIEDRIG)

Schweregrad: P3. Ein Nachzügler aus Track 18.

```
return NextResponse.json({ error: `Datenbankfehler: ${deleteError.message}` }, { status: 500 })
```

Track 18 hat rohe PostgREST-Meldungen an über 40 Stellen durch `dbError()` ersetzt — Tabellename, Spaltenname und Policy-Name gehören ins Log, nicht in die Antwort. Diese hier ist dabei durchgerutscht. Ebenso die Antwort des Abonnenten-Imports (`/api/admin/newsletter/subscribers/import`), die `error.message` roh weitergab.

Fix: `dbError('upload-delete', ...)`. Beim Import bleiben `inserted` und `skipped` in der Antwort — der Import läuft in Blöcken zu 500, ein Fehler im dritten Block lässt die ersten beiden **geschrieben** zurück, und diese Zahl braucht der Admin. Die Meldung selbst steht nur noch im Log.

Geprüft, ohne Befund

Bereich	Ergebnis
<code>isAuthorizedCron</code>	<code>timingSafeEqual</code> , fehlendes <code>CRON_SECRET</code> sperrt — sauber (Track 18)
<code>/api/cron/hard-delete</code>	Wiederholungsschutz (<code>.is('deleted_at', null)</code>) und PII-Entfernung greifen
<code>/api/cron/rental-payouts</code>	Track 16, unverändert
<code>/api/uploads</code> + <code>/api/uploads/[id]</code>	MIME-Positivliste, Größenbegrenzung, Besitzprüfung, privater Bucket mit frischer Signed URL je Aufruf; Zertifikate nur für Eigentümer und Admin
<code>/api/upload</code> (POST)	Besitzprüfung auf <code>salons.owner_id</code> , Bucket-Positivliste, Dateiendung aus dem geprüften MIME-Typ
<code>/api/admin/*</code>	jede Route prüft ihre Rolle selbst (<code>requireRole</code> bzw. eigener Guard); Middleware-RBAC deckt <code>/api/admin</code> zusätzlich ab
<code>/api/admin/newsletter/subscribers/import</code>	Zod-Schema, Obergrenze 5000, bestehende Adressen werden nicht überschrieben — eine Abmeldung bleibt also eine Abmeldung
<code>sendCampaign</code> EmpfängerAuswahl	<code>status = 'active'</code> , <code>exclude_tags</code> greifen
<code>/api/auth/phone/verify</code>	echter Versuchsähler seit Track 9, zeitkonstanter Vergleich; die bekannte Restlücke (Zähler pro Code braucht eine Migration) steht unverändert im Code
<code>/api/rental-listings</code>	<code>is_active</code> -Filter seit Track 15, Positivliste, Obergrenze 100
<code>/api/availability</code>	<code>salonAcceptsBusiness</code> , fail-closed bei Belegungsfehler
<code>/api/reviews/aggregate</code> Sichtbarkeitsregel	Miet-Bewertungen bleiben über <code>isSalonReview</code> ausgeschlossen
<code>sitemap.ts</code>	filtert durchgehend <code>is_active = true</code>
<code>/api/indexnow/key</code>	gibt nur den Verifizierungsschlüssel aus, der ohnehin öffentlich ist
<code>/api/analytics/vitals</code> , <code>/events</code> , <code>/visit</code>	Track 18/19, unverändert
<code>/api/messages</code> + <code>[conversationId]</code>	Teilnehmerprüfung vor jedem Lesen und Schreiben, 403 statt 404
<code>/api/notifications</code>	<code>user_id</code> -Filter auf Lesen und Schreiben, UUID-Prüfung, Obergrenze 100
<code>/api/calendar</code>	Berechtigungsprüfung (Kundin, Inhaber, Admin) und <code>attachmentDisposition</code> aus Track 19
<code>/api/me/payout-account</code>	volle IBAN verlässt den Server nie, nur die letzten vier Stellen

Änderungen

Datei	Änderung
src/lib/salon-status.ts	neu: salonIsPubliclyVisible()
src/app/api/salons/[id]/route.ts	404 für nicht freigegebene Salons; Positivlisten für services und rental_equipment
src/app/(public)/salon/[slug]/page.tsx	notFound() für nicht freigegebene Salons; keine Metadaten mehr für sie
src/lib/newsletter-sender.ts	CAS-Anspruch auf status ; Ergebnis-Code; keine rohen DB-Meldungen
src/app/api/admin/newsletter/campaigns/[id]/send/route.ts	404/409/500 statt pauschal 200
src/app/api/admin/newsletter/subscribers/import/route.ts	keine rohe DB-Meldung, Zahlen bleiben
src/app/api/analytics/meta-capi/route.ts	Ereignis- und Feld-Positivliste, Rate-Limit, keine Meta-Fehlerdetails, Prüfung vor dem Stub
src/app/api/auth/phone/send/route.ts	IP-Kontingent, Ländervorwahl-Positivliste
src/app/api/cron/publish-reviews/route.ts	rpc -Fehler werden gelesen und gezählt; 503 bei Abfragefehler
src/modules/reviews/review.service.ts	null für „nicht ermittelbar“; kein Überschreiben mit 0
src/modules/reviews/review.actions.ts	protokolliert einen nicht aktualisierten Schnitt
src/app/api/reviews/aggregate/route.ts	UUID-Prüfung, 503 statt erfundener Null
src/app/api/public-stats/route.ts	zählt nur Sichtbares
src/app/api/upload/[id]/route.ts	dbError()
src/__tests__/e2e/_harness/fake-supabase.ts	neu: rpc(), onRpc(), rpcCalls
src/__tests__/track-20-...test.ts	neu: 27 Tests

Tests

1430 → 1457 (27 neue), alle grün.

Gegenprobe gefahren: mit zurückgesetztem Produktivcode fallen **23 der 27** Tests durch. Die vier, die auch vorher grün sind, sind die Regressions-Wächter — freigeschalteter Salon bleibt sichtbar, `is_active = null` bleibt sichtbar, eine reguläre deutsche SMS-Anforderung geht durch, und der Cron bleibt ohne gültiges Secret verschlossen.

Gruppe	Tests
Öffentliche Salon-Sicht	5
Newsletter-Kampagne	4
meta-capi	4
SMS-Versand	3
publish-reviews	4
Bewertungs-Schnitt	4
public-stats	2
Bild-Löschen	1

Offen

- **Kampagne bleibt nach einem Timeout auf `sending` stehen** (siehe Befund 2). Braucht eine Entscheidung, ab wann ein Lauf als tot gilt.
- **`phone_verifications` hat keinen Zähler pro Code** — bekannt seit Track 9, braucht eine Migration, keine Codeänderung.
- **Die Salons mit `is_active = false` müssen einmal durchgesehen werden** (siehe Kasten unter Befund 1).